

Hoja de trabajo 2: Transformers y mecanismos de atención

Esteban Cárcamo (23016)

CC3092 Deep Learning y Sistemas Inteligentes, Universidad del Valle de Guatemala

5 de octubre de 2026

De las RNN a la atención

Cuello de botella. En un seq2seq con RNN el encoder comprime toda la oración en un único vector de tamaño fijo (su último estado oculto) y el decoder genera todo a partir de él. Cuanto más larga es la oración, más información debe caber en el mismo vector y más se diluyen las primeras palabras, así que la calidad cae con la longitud. La atención de Bahdanau et al. (2015) guarda *todos* los estados del encoder y, en cada paso, el decoder calcula un promedio ponderado de ellos (el contexto) según qué tan relevantes son para la palabra que va a generar.

Aditiva vs multiplicativa. Bahdanau puntúa con una pequeña red: $v^T \tanh(W_1 s + W_2 h)$. Luong et al. (2015) usan un producto punto, $s^T h$ (o $s^T W h$). La multiplicativa es más eficiente en hardware moderno porque todos los puntajes salen de una sola multiplicación de matrices QK^T , que las GPU ejecutan con núcleos especializados, sin la tanh ni la capa intermedia de la aditiva.

Q, K y V. Es una búsqueda «suave» en un diccionario: la consulta (query) se compara con todas las llaves (keys) y, en vez de devolver el valor de la llave exacta, devuelve un promedio de todos los valores (values) ponderado por la similitud. En *self-attention* Q, K y V son tres proyecciones lineales de la misma secuencia; en *cross-attention* Q sale del decoder y K y V salen de la salida del encoder.

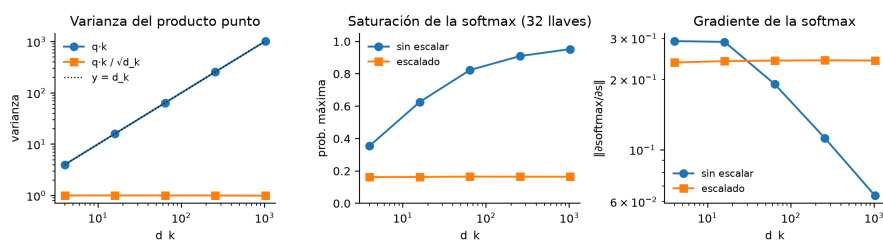
Qué elimina el Transformer. Sin recurrencia, todas las posiciones se procesan en paralelo (una RNN necesita n pasos secuenciales); cualquier par de tokens queda conectado en un solo paso (camino de longitud $O(1)$ en vez de $O(n)$); y como el gradiente no atraviesa una cadena de n multiplicaciones, desaparece el desvanecimiento a lo largo del tiempo.

La arquitectura Transformer

Scaled dot-product attention. Si los componentes de q y k son independientes con media 0 y varianza 1, $\text{Var}(q \cdot k) = \sum_i E[q_i^2] E[k_i^2] = d_k$. Sin escalar, con d_k grande los puntajes son enormes, la softmax se satura (casi toda la probabilidad en un token) y su jacobiano, $\text{diag}(p) - pp^T$, tiende a cero: el gradiente se desvanece. Dividir entre $\sqrt{d_k}$ devuelve la varianza a 1. La Figura 1 lo confirma: con $d_k = 1024$ la softmax sin escalar pone 0.95 en una sola llave y la norma del jacobiano cae a 0.06, contra 0.24 escalada.

Figura 1

Efecto del escalamiento por $\sqrt{d_k}$ (32 llaves, 2 000 consultas aleatorias)



Multi-head attention. Con h cabezas de d_{model}/h cada una aprende su propia proyección y puede atender a un tipo de relación distinto (sintaxis, posición, correferencia) a la vez; una sola cabeza promediaría todo en una única distribución. El número de parámetros no cambia: $4d^2 + 4d = 1\,050\,624$ con $d = 512$ para 1, 8 o 16 cabezas (verificado en el notebook).

Codificación posicional. La self-attention es equivariante a permutaciones: si se reordenan los tokens, las salidas se reordenan igual, así que no sabe el orden. La codificación sinusoidal suma a cada embedding $\text{PE}(\text{pos}, 2i) = \sin(\text{pos}/10000^{2i/d})$ y $\text{PE}(\text{pos}, 2i+1) = \cos(\dots)$, con frecuencias geométricas; es fija y un desplazamiento relativo equivale a una transformación lineal. Los embeddings aprendidos (BERT, GPT-2) son

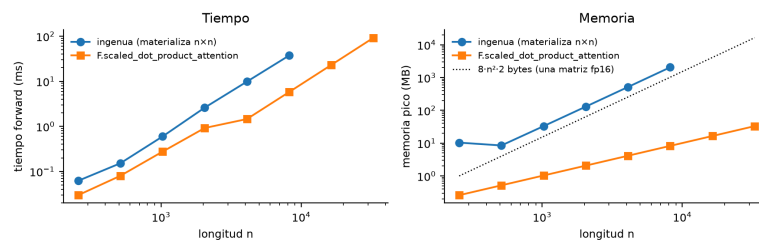
una tabla entrenable que no sirve más allá de la longitud máxima (512 o 1 024). RoPE (Su et al., 2021) rota q y k según la posición, de modo que $q \cdot k$ depende solo de la distancia relativa; es la que usan LLaMA y la mayoría de los modelos actuales.

Bloque completo. Cada subcapa (atención y FFN) va envuelta en una conexión residual, $x + F(x)$, que deja pasar el gradiente, y una LayerNorm, que estabiliza la escala de las activaciones. La FFN ($d \rightarrow 4d \rightarrow d$ con una no linealidad) se aplica igual a cada posición. Post-LN normaliza después de sumar, $\text{LN}(x + F(x))$; Pre-LN normaliza antes, $x + F(\text{LN}(x))$, y deja el camino residual como una identidad limpia. Al inicializar 24 capas, en Post-LN la última recibe 7.6 veces más gradiente que la primera, lo que obliga a usar *warm-up*; Pre-LN entrena estable sin él (Xiong et al., 2020).

Complejidad. La self-attention cuesta $O(n^2 \cdot d)$ en tiempo y $O(n^2)$ en memoria por cabeza, porque materializa la matriz $n \times n$. En la Figura 2 la implementación ingenua crece con pendiente ≈ 2 en ambos ejes y se queda sin memoria con $n = 16\,384$ en una GPU de 6 GB. **FlashAttention** (Dao et al., 2022) calcula la softmax por bloques en la SRAM sin guardar la matriz: es exacta y deja la memoria en $O(n)$ (33 MB con $n = 32\,768$), pero el tiempo sigue siendo cuadrático y requiere kernels específicos. La **atención por ventanas** (Beltagy et al., 2020) solo mira w vecinos, $O(n \cdot w)$, y sacrifica las dependencias directas lejanas. La **atención lineal** (Katharopoulos et al., 2020) cambia la softmax por un kernel $\phi(q)\phi(k)^T$ y reordena el producto, $O(n \cdot d^2)$, a costa de una atención menos precisa.

Figura 2

Tiempo y memoria de la atención frente a la longitud (RTX 4050, fp16, 8 cabezas, $d_k = 64$)



Atención en PyTorch y Hugging Face

Clase o función	Propósito y parámetros relevantes
<code>F.scaled_dot_product_attention</code>	Atención fusionada; elige FlashAttention o memory-efficient si puede. <code>attn_mask</code> : booleana (True = atender) o aditiva; <code>dropout_p</code> : dropout sobre los pesos (solo en entrenamiento); <code>is_causal</code> : máscara triangular sin construirla; <code>scale</code> : reemplaza $1/\sqrt{d_k}$. No devuelve los pesos.
<code>nn.MultiheadAttention</code>	<code>embed_dim</code> y <code>num_heads</code> (cada cabeza mide <code>embed_dim/num_heads</code>); <code>batch_first</code> : entrada (B, n, d) en vez de (n, B, d); <code>kdim</code> , <code>vdim</code> : tamaño de K y V si vienen de otra fuente (cross-attention). Forward: <code>key_padding_mask</code> (B, n) oculta el padding, <code>attn_mask</code> (n, n) oculta pares (True = no atender), <code>need_weights</code> devuelve los pesos y <code>average_attn_weights</code> los promedia entre cabezas.
<code>nn.TransformerEncoderLayer</code>	Bloque completo: <code>d_model</code> , <code>nhead</code> , <code>dim_feedforward</code> (ancho de la FFN), <code>dropout</code> y <code>norm_first</code> (True = Pre-LN). <code>nn.Transformer.generate_square_subsequent_mask(n)</code> crea la máscara causal: 0 en y bajo la diagonal y $-\infty$ encima.
<code>output_attentions=True</code>	Hugging Face devuelve <code>outputs.attentions</code> : un tensor por capa de forma <code>batch × cabezas × n × n</code> (12 capas × 12 cabezas en BERT-base y GPT-2). Requiere <code>attn_implementation="eager"</code> : los kernels fusionados no calculan los pesos.

Ejercicio a mano y verificaciones

Tres tokens con $d = d_k = 2$: $X = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}$, $W_Q = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}$, $W_K = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$ y $W_V = \begin{bmatrix} 1 & 0 \\ 0 & 2 \end{bmatrix}$. Se obtienen $Q = XW_Q$, $K = XW_K$ y $V = XW_V$; para cada fila de QK^T se divide entre $\sqrt{2} =$

1.414, se aplica la exponencial, se normaliza (softmax) y se calcula la salida $\sum_j a_{ij}v_j$. En la tabla, k y v de la fila j son los del token j .

Token	q	k	v	$q \cdot k_{1..3}$	$\div \sqrt{2}$	exp (suma)	Pesos a	Salida
1	[1, 0]	[1, 1]	[1, 0]	1, 0, 1	0.707, 0, 0.707	2.03, 1.00, 2.03 (5.06)	0.401, 0.198, 0.401	[0.802, 1.198]
2	[1, 1]	[0, 1]	[0, 2]	2, 1, 3	1.414, 0.707, 2.121	4.11, 2.03, 8.34 (14.48)	0.284, 0.140, 0.576	[0.860, 1.432]
3	[2, 1]	[1, 2]	[1, 2]	3, 1, 4	2.121, 0.707, 2.828	8.34, 2.03, 16.92 (27.29)	0.306, 0.074, 0.620	[0.926, 1.389]

Por ejemplo, la salida del token 2 es $0.284 \cdot [1, 0] + 0.140 \cdot [0, 2] + 0.576 \cdot [1, 2] = [0.860, 1.432]$. El token 3 atiende sobre todo a k_3 , la llave más alineada con q_3 . Con máscara causal las filas 1 y 2 quedan $[1, 0, 0]$ y $[0.670, 0.330, 0]$; la 3 no cambia. El código da lo mismo.

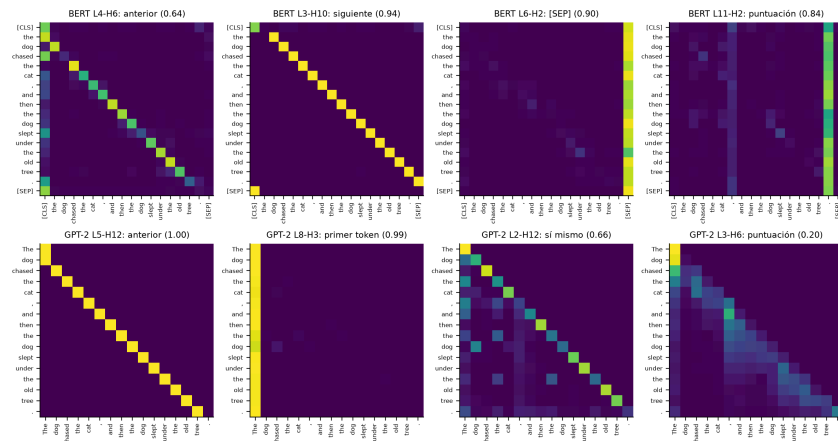
Verificaciones. La implementación propia coincide con `F.scaled_dot_product_attention` sin máscara, con `is_causal`, con una `attn_mask` de padding y con `scale = 0.5` (error máximo 4.8×10^{-7}). La multi-head propia, con los pesos copiados de `nn.MultiheadAttention`, reproduce sus salidas y sus pesos por cabeza con máscara causal y `key_padding_mask` (error 1.2×10^{-7}).

Atención en BERT y GPT-2

Patrones por cabeza. Para cada una de las 144 cabezas se midió, en ocho oraciones, la atención media al token anterior, al siguiente, a sí mismo, a [CLS] o primer token, a [SEP], a la puntuación y a otras apariciones de la misma palabra. La Figura 3 muestra la cabeza más representativa de cada patrón en «The dog chased the cat, and then the dog slept under the old tree.» Como describen Clark et al. (2019), hay cabezas posicionales casi perfectas (BERT L3-H10 mira al siguiente token con 0.94; GPT-2 L5-H12, al anterior con 1.00), cabezas que mandan todo a [SEP] (41 de 144 en BERT le dan más de la mitad) o al punto final, y en GPT-2 cabezas que ponen el 99 % en el primer token (L8-H3). Además, BERT L2-H12 conecta «dog» con el otro «dog» (0.48 en promedio).

Figura 3

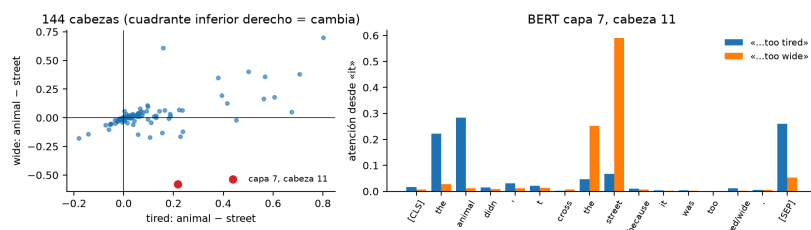
Mapas de atención de cuatro cabezas de capas distintas en BERT (arriba) y GPT-2 (abajo); filas = consulta, columnas = llave



«it» → «animal» o «street». En 22 de las 144 cabezas de BERT «it» atiende más a «animal» con *tired* y más a «street» con *wide*. La más clara es L7-H11: con *tired* da 0.28 a «animal» y 0.07 a «street»; con *wide*, 0.01 y 0.59 (Figura 4). La mayoría de las cabezas, en cambio, prefiere «animal» en ambos casos (cuadrante superior derecho). En GPT-2 esto no puede pasar: «it» aparece antes de *tired/wide* y la máscara causal se lo oculta.

Figura 4

Correferencia en BERT: diferencia animal – street por cabeza en las dos oraciones (izquierda) y atención desde «it» en L7-H11 (derecha)

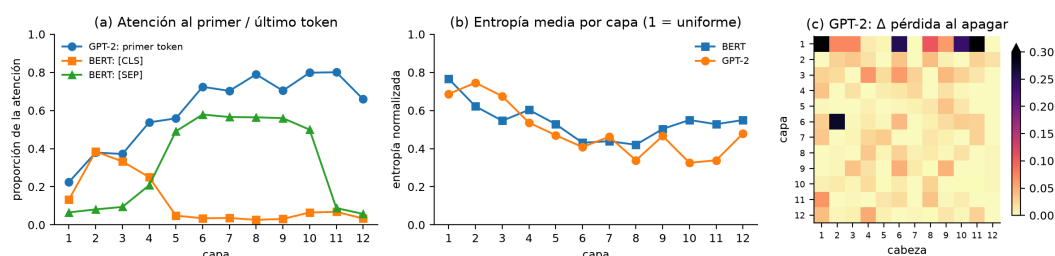


GPT-2 causal y attention sinks. En todas las capas y cabezas el valor máximo por encima de la diagonal es exactamente 0 y cada fila suma 1: la matriz es triangular inferior. El primer token recibe el 22 % de la atención en la capa 1 y entre el 70 y el 80 % en las capas 6 a 11 (Figura 5a). No es por su contenido: cualquier palabra en la primera posición («The», «Paris», «Water», ...) recibe ≈ 0.53 , y esa misma palabra más adelante solo ≈ 0.05 . Es un *attention sink* (Xiao et al., 2023): como la softmax obliga a repartir una masa de 1, las cabezas que no necesitan información la «estacionan» en un token que todas las posiciones ven y cuyo valor aporta poco (por eso StreamingLLM conserva esos tokens en la caché KV). BERT hace lo mismo con [CLS] (capas 2–4) y [SEP] (capas 5–10).

Entropía por capa. Normalizada (1 = uniforme), baja de 0.77 en la capa 1 a 0.42 en la 8 en BERT y de 0.69 a 0.34 en GPT-2 (Figura 5b): las capas profundas atienden de forma **más concentrada**. BERT vuelve a abrirse en las últimas capas (0.55). Gran parte de esa concentración es el sink. Al apagar cabezas de GPT-2 una por una (Figura 5c), 71 de 144 cambian la pérdida en menos de 0.01 y solo 6 la suben más de 0.1, mientras que L1-H1 y L1-H11 la suben 2.5 y 2.1 nats.

Figura 5

Atención al primer token, entropía por capa y ablación de cabezas en GPT-2 (Δ pérdida sobre un texto de 161 tokens)



Discusión

¿Por qué la atención permitió modelos más grandes? Una capa procesa los n tokens con unas pocas multiplicaciones de matrices en paralelo, en lugar de n pasos secuenciales, y aprovecha GPU con miles de núcleos; además, el camino entre dos tokens es de un paso, así que las dependencias lejanas se aprenden sin desvanecimiento. El precio es el costo cuadrático, aceptable mientras n no supere mucho a d (n^2d frente a nd^2 de una RNN).

¿Se especializan las cabezas? ¿Todas son útiles? Sí hay especialización: cabezas posicionales, de separadores, de puntuación, de la misma palabra y de correferencia (L7-H11). Pero no todas son útiles por separado: la mitad casi no afecta la pérdida y dos cabezas de la capa 1 concentran el efecto. Aun así, la redundancia tiene un límite: en un texto distinto, apagar el 10, 25 y 50 % de las menos importantes sube la perplejidad de 23.9 a 26.9, 30.9 y 46.1, en línea con Michel et al. (2019).

¿Los pesos de atención explican las decisiones? Solo en parte. Jain y Wallace (2019) muestran que los pesos se correlacionan poco con la importancia medida por gradientes y que distribuciones muy distintas dan la misma predicción. Wiegrefe y Pinter (2019) responden que esas distribuciones no las produce el modelo entrenado y que la atención puede ser una explicación *plausible*, aunque no necesariamente *fidel*. Nuestros

resultados apoyan esa postura intermedia: L7-H11 ofrece una explicación plausible de la correferencia, pero es una cabeza de 144, y en GPT-2 el 70–80 % de la atención va a un token sin contenido, así que un peso alto no significa importancia. Para afirmar causalidad se necesitan intervenciones como la ablación.

¿Cuándo preferir una RNN o una CNN? Con datos en *streaming* que necesitan memoria constante por paso (RNN o modelos de estado como Mamba), en dispositivos con poca memoria o latencia estricta, y con pocos datos, donde el sesgo inductivo de la CNN (localidad e invariancia a traslaciones en imágenes o audio) generaliza mejor.

Documentos de 100 000 tokens. La matriz de una sola cabeza tendría 10^{10} entradas (20 GB en fp16), por cada cabeza y cada capa. FlashAttention resuelve la memoria, pero no el tiempo: extrapolando la Figura 2, ≈ 0.9 s por capa solo en la atención. Consideraría atención por ventanas con tokens globales (Longformer), atención lineal o modelos de estado, RoPE con interpolación para extender el contexto, o partir el documento y recuperar solo los fragmentos relevantes (RAG).

Conclusiones

- La atención implementada desde cero coincide con la de PyTorch (error $\approx 10^{-7}$) y con el ejercicio a mano.
- Dividir entre $\sqrt{d_k}$ mantiene la varianza de los puntajes en 1 y evita que la softmax se sature y anule el gradiente.
- La atención estándar es $O(n^2)$ en tiempo y memoria; FlashAttention deja la memoria en $O(n)$, pero no el tiempo.
- Las cabezas se especializan, pero muchas son redundantes, y las capas profundas concentran la atención en sinks (primer token o [SEP]).
- Los pesos de atención sirven para formular hipótesis sobre el modelo, no como explicación fiel de sus decisiones.

Referencias

- Bahdanau, D., Cho, K., & Bengio, Y. (2015). Neural machine translation by jointly learning to align and translate. *ICLR*. <https://arxiv.org/abs/1409.0473>
- Beltagy, I., Peters, M. E., & Cohan, A. (2020). *Longformer: The long-document transformer*. <https://arxiv.org/abs/2004.05150>
- Clark, K., Khandelwal, U., Levy, O., & Manning, C. D. (2019). What does BERT look at? An analysis of BERT's attention. *BlackboxNLP*, 276–286.
- Dao, T., Fu, D. Y., Ermon, S., Rudra, A., & Ré, C. (2022). FlashAttention: Fast and memory-efficient exact attention with IO-awareness. *NeurIPS*.
- Jain, S., & Wallace, B. C. (2019). Attention is not explanation. *NAACL-HLT*, 3543–3556.
- Katharopoulos, A., Vyas, A., Pappas, N., & Fleuret, F. (2020). Transformers are RNNs: Fast autoregressive transformers with linear attention. *ICML*.
- Luong, M.-T., Pham, H., & Manning, C. D. (2015). Effective approaches to attention-based neural machine translation. *EMNLP*, 1412–1421.
- Michel, P., Levy, O., & Neubig, G. (2019). Are sixteen heads really better than one? *NeurIPS*.
- Su, J., Lu, Y., Pan, S., Murtadha, A., Wen, B., & Liu, Y. (2021). *RoFormer: Enhanced transformer with rotary position embedding*. <https://arxiv.org/abs/2104.09864>
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. *NeurIPS*. <https://arxiv.org/abs/1706.03762>
- Wiegrefe, S., & Pinter, Y. (2019). Attention is not not explanation. *EMNLP-IJCNLP*, 11–20.
- Xiao, G., Tian, Y., Chen, B., Han, S., & Lewis, M. (2023). *Efficient streaming language models with attention sinks*. <https://arxiv.org/abs/2309.17453>
- Xiong, R., Yang, Y., He, D., Zheng, K., Zheng, S., Xing, C., Zhang, H., Lan, Y., Wang, L., & Liu, T. (2020). On layer normalization in the transformer architecture. *ICML*.

Repositorio: <https://github.com/ecarcamo/CC3092-DEEP-LEARNING/tree/main/HTD2>